Laboratório Funcional · Aula 12

Luiz Otavio Roder · RA 24159236-2 · Turma ESOFT-C
Gerado em 19/10/2026, 19:24:18 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora() é impura porque usa datetime.now(): o resultado depende do relógio, que muda entre chamadas, mesmo sem parâmetros. poe(lista, x) também é impura porque lista.append(x) modifica a lista recebida, produzindo um efeito colateral fora da função. Uma função pura precisa depender somente das entradas e não modificar estado observável.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: 1: A-1 A-2, porque proximo_id altera o contador global a cada chamada. Linha 4: 4: 14, pois g(4) = 12 e f(12) = 14. Linha 7: 7: [7, 3, 9] [3, 7, 9], pois sorted cria outra lista sem alterar notas.

Saídas (original e com compor(g, f))

```
EXECUÇÃO ORIGINAL (Python — verificada):
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]

APÓS MUDAR A LINHA 4 PARA compor(g, f)(4):
1: A-1 A-2
```

```
2: True
3: 8
4: 18
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
(A segunda saída difere apenas na linha 4; valor calculado a partir da alteração solicitada.)
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

proximo_id("A") incrementa a variável global contador a cada chamada. Assim, com o mesmo argumento "A", a primeira chamada retorna "A-1" e a segunda, "A-2". Seu resultado depende do histórico de execução, não somente do parâmetro. Nos testes é preciso controlar ou reinicializar esse estado global; um teste pode interferir em outro e mudar de resultado conforme a ordem.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

f(x) = x + 2 e g(x) = 3x. compor(f, g)(4) = f(g(4)) = f(12) = 14. Já compor(g, f)(4) = g(f(4)) = g(6) = 18. A ordem importa: composição de funções, em geral, não é comutativa.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) retorna uma nova lista ordenada ([3, 7, 9]) sem modificar a lista original ([7, 3, 9]). Já notas.sort() ordena a própria lista e retorna None, alterando seu conteúdo. sorted é mais próximo do estilo funcional porque evita mutação e preserva o valor anterior.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
SAÍDA ESPERADA DO CÓDIGO ORIGINAL (Scheme, deduzida das expressões):
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B

AO TROCAR (car '(A B C)) POR (car (A B C)):
As cinco primeiras linhas ainda são 22, 14, 25, 8 e negativo. Em seguida ocorre erro de
avaliação: (A B C) é tratado como chamada de função, mas A não foi definido como
procedimento. As linhas posteriores não são executadas.
```

```
(A mensagem exata de erro depende do interpretador Scheme; não houve execução deste
ambiente.)
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo é uma abreviação de quote: impede que o Scheme avalie a lista como código e faz dela um dado literal. Sem ele, (A B C) é interpretado como chamada da função A com os argumentos B e C; como A não é uma função definida, ocorre erro antes de car receber uma lista.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
Expressão: (car (cdr (cdr '(A B C))))
Resultado esperado: C
Primeiro cdr remove A, o segundo remove B, e car seleciona C.
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

No Scheme, espero que a função em cauda conta-ate consiga chegar a 1.000.000 sem esgotar a pilha, pois a implementação pode reutilizar a ativação da chamada anterior. No Python, a mesma recursão deve funcionar para 500, mas gerar RecursionError em 1.000.000, porque Python não elimina chamadas em cauda e tem limite de profundidade.

Saída do Scheme

```
SAÍDA ESPERADA (Scheme — valores determinados pelo código):
#t
#f
2432902008176640000
2432902008176640000
1000000
(Saída calculada; execução em Scheme não disponível neste ambiente.)
```

Saída do Python (as últimas linhas bastam)

```
SAÍDA REAL (Python — verificada):
500
Traceback (most recent call last):
  File "...", line 9, in <module>
    print(conta_ate(1_000_000, 0))
  File "...", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
(Caminho do arquivo e número de repetições podem variar conforme a versão do Python.)
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, crescem a pilha de chamadas e as multiplicações pendentes: cada fatorial(n) precisa esperar o retorno de fatorial(n-1) para multiplicar por n. Na recursão em cauda, o resultado parcial fica no acumulador e a chamada recursiva é a última operação, sem trabalho pendente. Uma implementação com otimização de cauda pode reutilizar o mesmo registro de ativação, mantendo o espaço de pilha constante.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme exige suporte a chamadas em cauda próprias: conta-ate pode fazer 1.000.000 de passos reaproveitando o registro de ativação e sem aumentar a pilha. O Python não realiza essa otimização; cada chamada recursiva cria um novo quadro, até atingir o limite de recursão e gerar RecursionError: maximum recursion depth exceeded.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
SAÍDA ESPERADA APÓS SUBSTITUIR AS TRÊS DEFINIÇÕES (Scheme):
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
(Valores determinados pelo código das funções; interpretador Scheme não executado neste
ambiente.)
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte que exige mais atenção é montar corretamente o caso base e a chamada recursiva. Em tamanho, null? devolve 0 para a lista vazia e cdr passa o restante da lista para contar; car não é necessário, pois só contamos elementos. Em soma-quadrados, null? encerra com 0, car fornece o número a ser elevado ao quadrado e cdr continua a soma no restante. Em pares, null? devolve a lista vazia, car verifica se o número é par e, quando é, entra em cons; cdr avança para os demais elementos.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares deve imprimir [0,1,4,9,16], porque só exige os cinco primeiros quadrados da lista infinita. primeiro 42 (div 1 0) deve imprimir 42 sem calcular a divisão por zero, já que o segundo argumento não é usado e Haskell faz avaliação preguiçosa.

Saídas (original e com member 17)

```
SAÍDA ESPERADA DO CÓDIGO ORIGINAL (Haskell, deduzida das expressões):
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

AO DESCOMENTAR print (member 17 squares):
As saídas anteriores aparecem, mas essa última busca não termina: 17 não é quadrado perfeito
e member percorre a lista infinita sem condição para parar. O ambiente de execução pode
interromper o programa por limite de tempo ou memória.
(Saídas determinadas pelo código; não executei Haskell neste ambiente.)
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

Haskell avalia as expressões por demanda (avaliação preguiçosa): uma lista pode ser descrita infinitamente, mas take 5 força a produção de apenas cinco elementos. A função primeiro x _ = x não usa o segundo argumento, então (div 1 0) nunca é avaliado; por isso a chamada devolve 42 sem divisão por zero.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 16 squares termina com True porque 16 aparece na sequência (0, 1, 4, 9, 16, ...). member 17 squares não termina: 17 não pertence à lista infinita, e a função não possui critério de parada para rejeitar valores ausentes. Como squares é crescente, poderíamos encerrar ao ultrapassar o valor procurado. Por exemplo:

```
memberOrd _ [] = False
memberOrd b (a:xs)
  | a == b    = True
  | a > b     = False
  | otherwise = memberOrd b xs
```

Assim, memberOrd 17 squares termina com False ao chegar a 25.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) imprime 15, porque a função devolvida soma 10 a 5. [1, 2, 3].map(vezes(5)) imprime [5, 10, 15], porque vezes(5) devolve uma função que multiplica cada elemento por 5.

Saída do JavaScript

```
SAÍDA REAL (JavaScript/Node.js — verificada):
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
SAÍDA REAL (Java — verificada):
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função interna forma um fechamento (closure) com o ambiente léxico onde foi criada. Quando somador(10) retorna essa função, ela mantém acesso à variável capturada n, cujo valor é 10, mesmo após o término da chamada externa. Por isso mais10(5) calcula 5 + 10 = 15.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Em Java, f.compose(g).apply(4) significa f(g(4)): primeiro g(4) = 12, depois f(12) = 14. Já f.andThen(g).apply(4) é g(f(4)): primeiro f(4) = 6, depois g(6) = 18. f.compose(g) corresponde a compor(f, g) da Prática 1; andThen inverte a ordem.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
// Exercício 1 — Java (Stream, sem laço nem acumulador explícito)
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();
        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
// Exercício 2 — JavaScript (filter + map, sem laço nem push)
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];
```

```
const aprovados = alunos
  .filter(aluno ⇒ aluno.nota ⩾ 6)
  .map(aluno ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

```
SAÍDAS REAIS DAS VERSÕES FUNCIONAIS (executadas):
Exercício 1 — Java: 244
Exercício 2 — JavaScript: ANA, CARLA, DAVI

Conferência: 8² + 12² + 6² = 64 + 144 + 36 = 244. A lista de aprovados inclui apenas notas ⩾
6.
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

Para esses exercícios, considero a versão funcional mais fácil de ler: a sequência filter → map → sum descreve diretamente as transformações, sem controlar índices nem acumuladores. A versão com laço pode ser mais simples de depurar passo a passo, porque é fácil parar dentro de cada iteração e inspecionar variáveis. Na versão funcional também se pode depurar, inspecionando o resultado de cada etapa ou usando ferramentas de depuração.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais compacta e expressa a intenção diretamente: gerar números, elevar ao cubo e somar. A versão em C pode ser mais familiar para quem aprendeu primeiro os laços. Aceitaria um possível custo de desempenho em relatórios, sistemas administrativos e aplicações em que clareza, segurança e manutenção são mais importantes do que alguns milissegundos. Em sistemas de tempo real com prazos rigorosos, processamento de alta frequência ou código com restrição severa de recursos, mediria o desempenho e priorizaria cumprir os requisitos. A comparação do livro é uma estimativa contextual, não uma regra universal.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi conseguir trabalhar com listas infinitas em Haskell sem calcular todos os elementos, graças à avaliação preguiçosa.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro de impressão está em [b | b <- t, b <- h]: o segundo <- é um gerador, mas h é um elemento (o pivô), não uma lista. O correto é o filtro b <= h: sort (h:t) = sort [b | b <- t, b <= h] ++ [h] ++ sort [b | b <- t, b > h]. O compilador aponta uma incompatibilidade de tipos, porque tenta usar h como uma coleção de onde extrair elementos, embora o padrão (h:t) faça de h um elemento de t. A redação exata do erro varia com a versão do compilador Haskell.